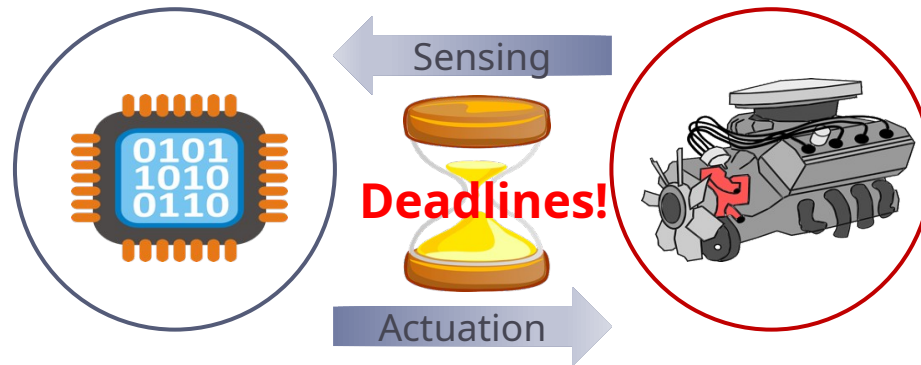


WCET-Aware Stack Frame Management of Embedded Systems using Scratchpad Memories

Yooseong Kim, Mohammad Khayatian, **Aviral Shrivastava**
Arizona State University

Hard Real-time Systems

- ▶ Strict timing constraints
 - ▶ Tasks have deadlines to finish execution



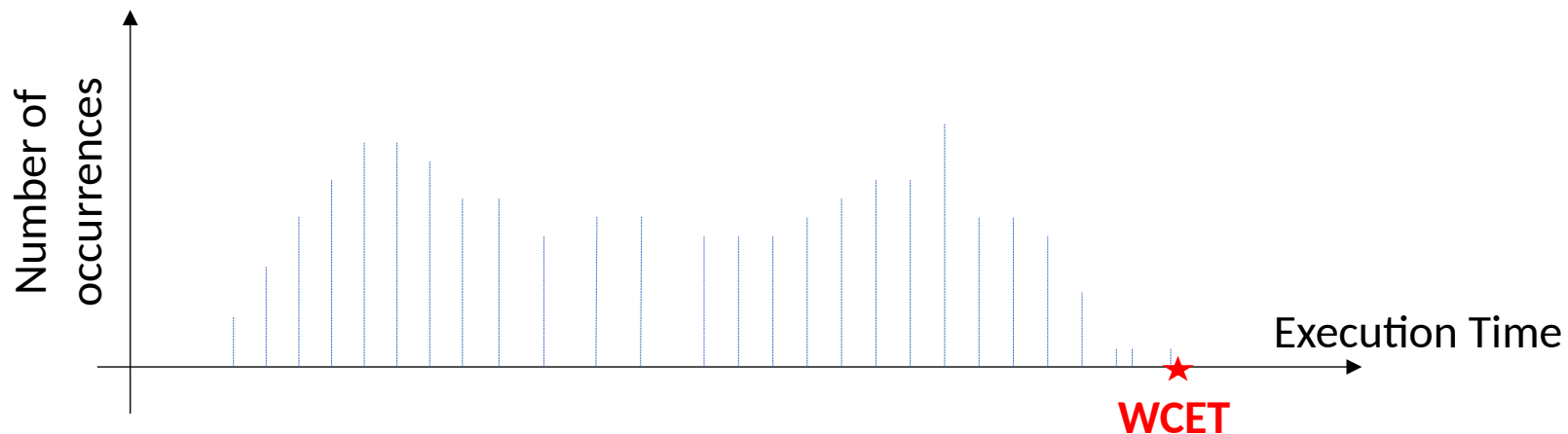
- ▶ Timing determines correctness, not just performance
 - ▶ Missing a deadline can be catastrophic



Image credit: JasonParis@Flickr

Ensuring Timing Correctness

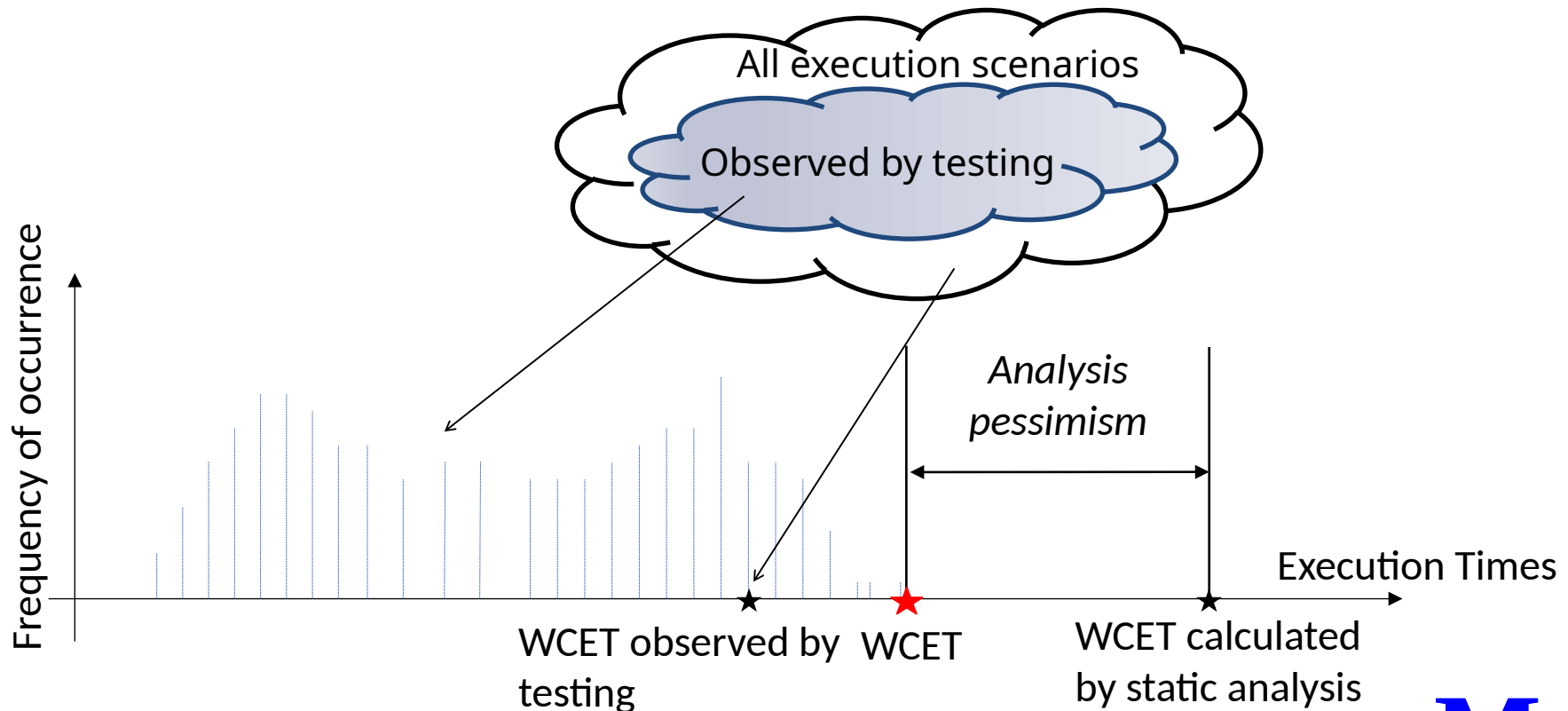
- ▶ Guarantee the absence of missed deadlines
 - ▶ Worst-case execution time (WCET) analysis



- ▶ How?
 - ▶ Testing based on measurement
 - ▶ No Guarantee!
 - ▶ Static analysis
 - ▶ Use structural information of HW/SW

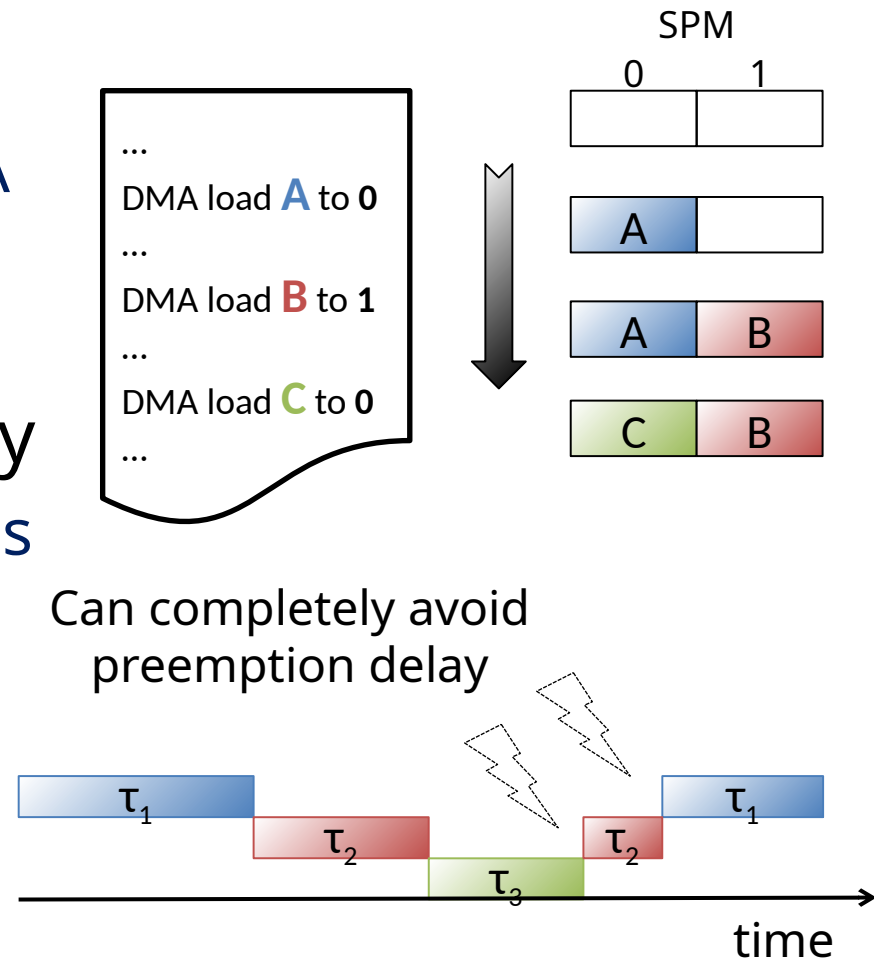
Analyses

- ▶ WCET is Pessimistic
 - ▶ Microarchitectural state uncertainties
 - ▶ Cache, branch predictors, pipelines, ...



Scratchpad Memories (SPM) . A Promising Alternative

- ▶ Raw directly-addressable SRAM
 - ▶ Explicitly controlled by DMA (Direct Memory Access)
- ▶ Provide time predictability
 - ▶ Determine a memory access is a hit or miss



Stack Frame Management

- ▶ Perform code transformation
 - ▶ Store frame stack in SPM before a function call
 - ▶ Load frame stack from SPM after a function call

```
DMA_store();  
Sp_reset();
```

Inserted code *before* a call

```
// DMA stores the stack in the main memory  
// reset the stack pointer
```

```
Foo();
```

```
// call a function
```

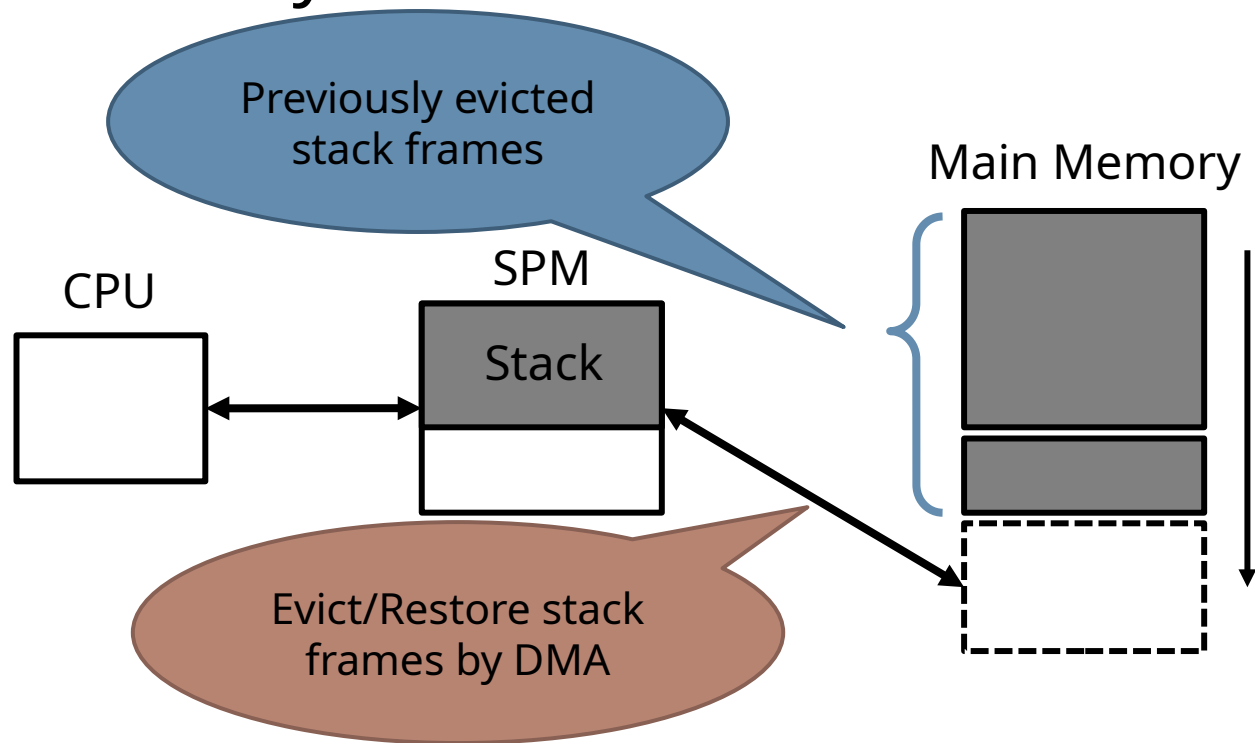
```
DMA_load();  
Sp_restore();
```

```
// DMA loads the stack from the main memory  
// restore the stack pointer
```

Inserted code *after* a call

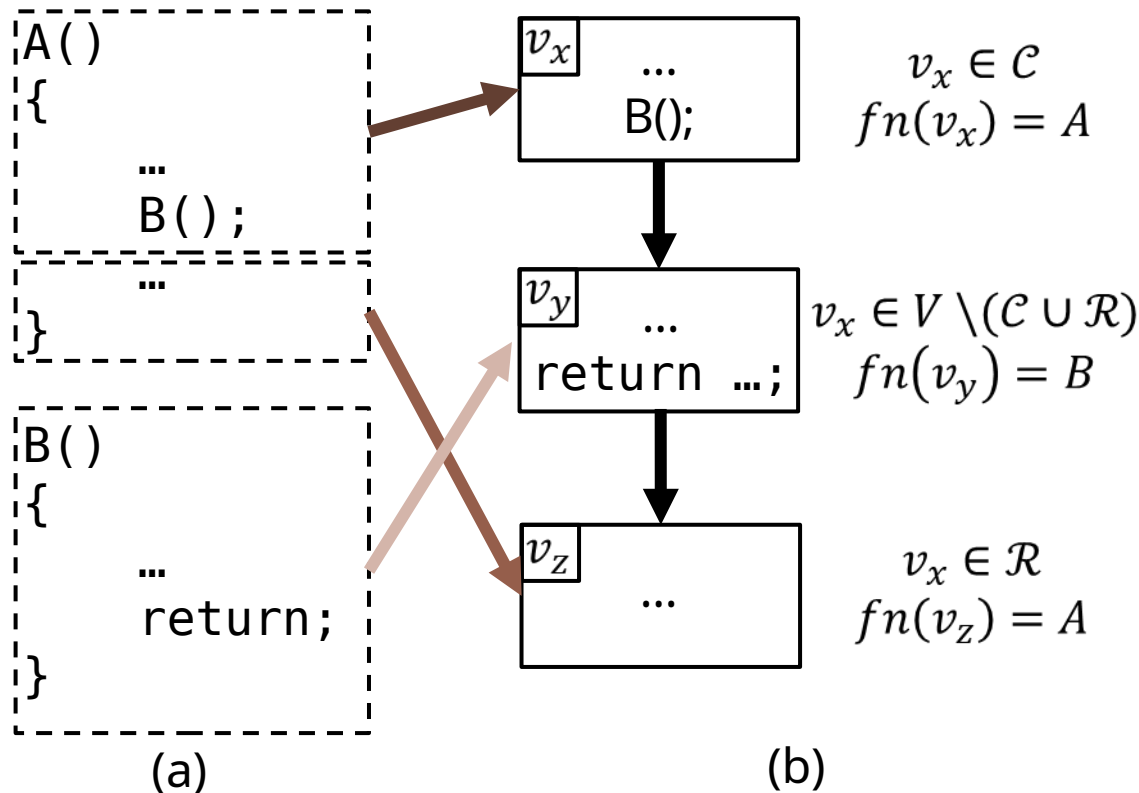
Virtual Stack in Main Memory

- ▶ Moving stack frames needs previous execution history



Problem Formulation from CFG

- ▶ Use inline Control Flow Graph (CFG) presentation
 - ▶ V is set of all basic blocks ($v_x, v_y, v_z \in V$)
 - ▶ $\mathcal{C}$ and $\mathcal{R}$ are set of basic blocks containing Call and Return



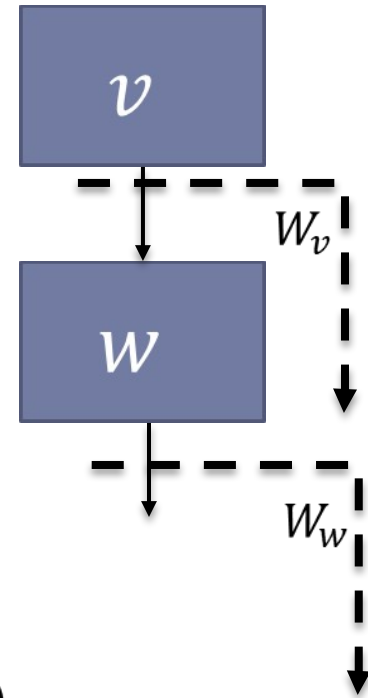
Problem

- ▶ Minimize the WCET (W_{v_s}) of the whole program.

$$\min W_{v_s}$$

- ▶ v_s and v_t are the start and final basic blocks
- ▶ n_v execution frequency of basic block v
- ▶ C_v Management cost of basic block v
- ▶ t_v execution time of basic block v
- ▶ E is set of edges

$$\forall (v, w) \in E, \quad W_v \geq W_w + n_v \cdot (t_v + C_v)$$
$$W_{v_t} = n_{v_t} \cdot (t_{v_t} + C_{v_t})$$



Decision Variable

- ▶ Management Cost (C_v):

$$\forall v \in \mathcal{C}, \quad C_v = \begin{cases} mo_c(S_v) & \text{if } M_v = 1 \\ 0 & \text{if } M_v = 0 \end{cases}$$
$$\forall v \in \mathcal{R}, \quad C_v = \begin{cases} mo_r(S_v) & \text{if } M_{cl(v)} = 1 \\ 0 & \text{if } M_{cl(v)} = 0 \end{cases}$$

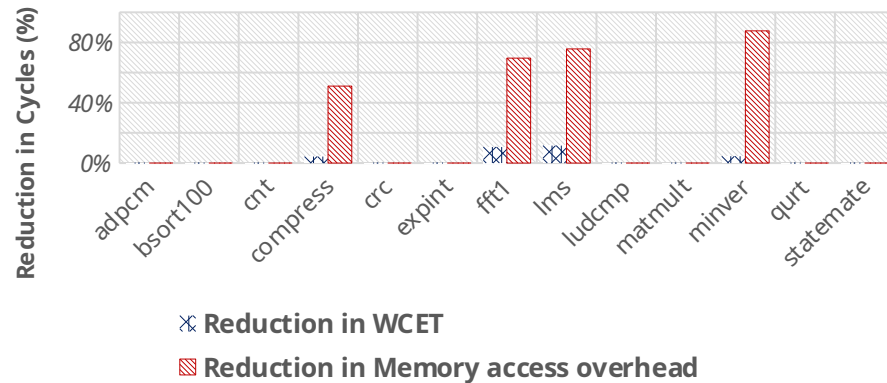
- ▶ $mo_c(x), mo_r(x)$ overheads of evicting and restoring
- ▶ M_v decision variable
- ▶ $M_{cl(c)}$ decision variable of the called basic block

Testbed

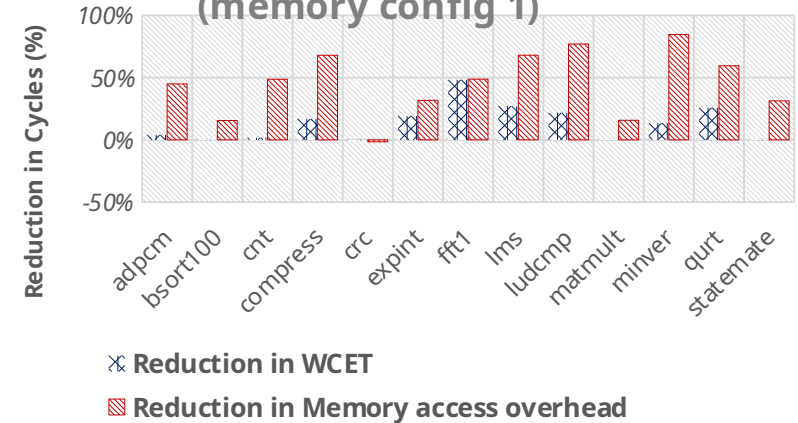
- ▶ 13 Benchmarks from Mälardalen WCET suite
- ▶ ARMv4 ISA
- ▶ SPM size of two memory configurations
 - ▶ $m + 0.5(M - m)$
 - ▶ $m + 0.7(M - m)$
- ▶ m and M are the min and max stack frames

WCET and Memory Overhead

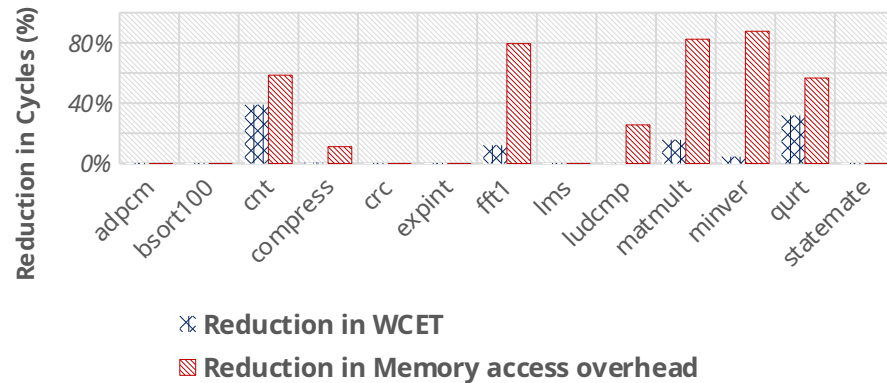
Comparing with Lu et al. (memory config 1)



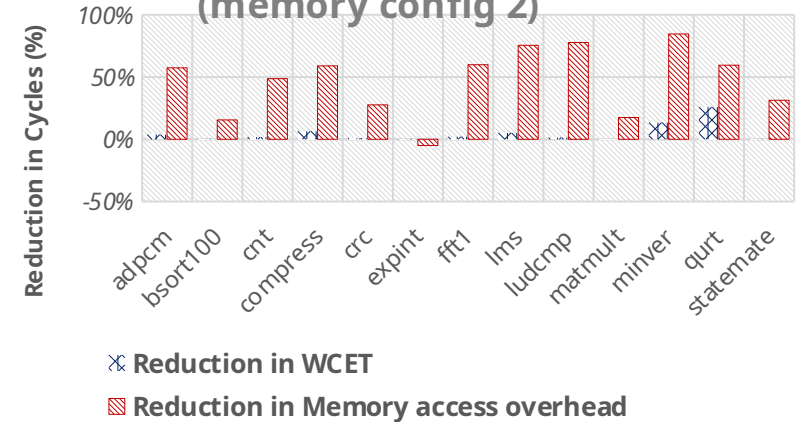
Comparing with Liu and Zhang (memory config 1)



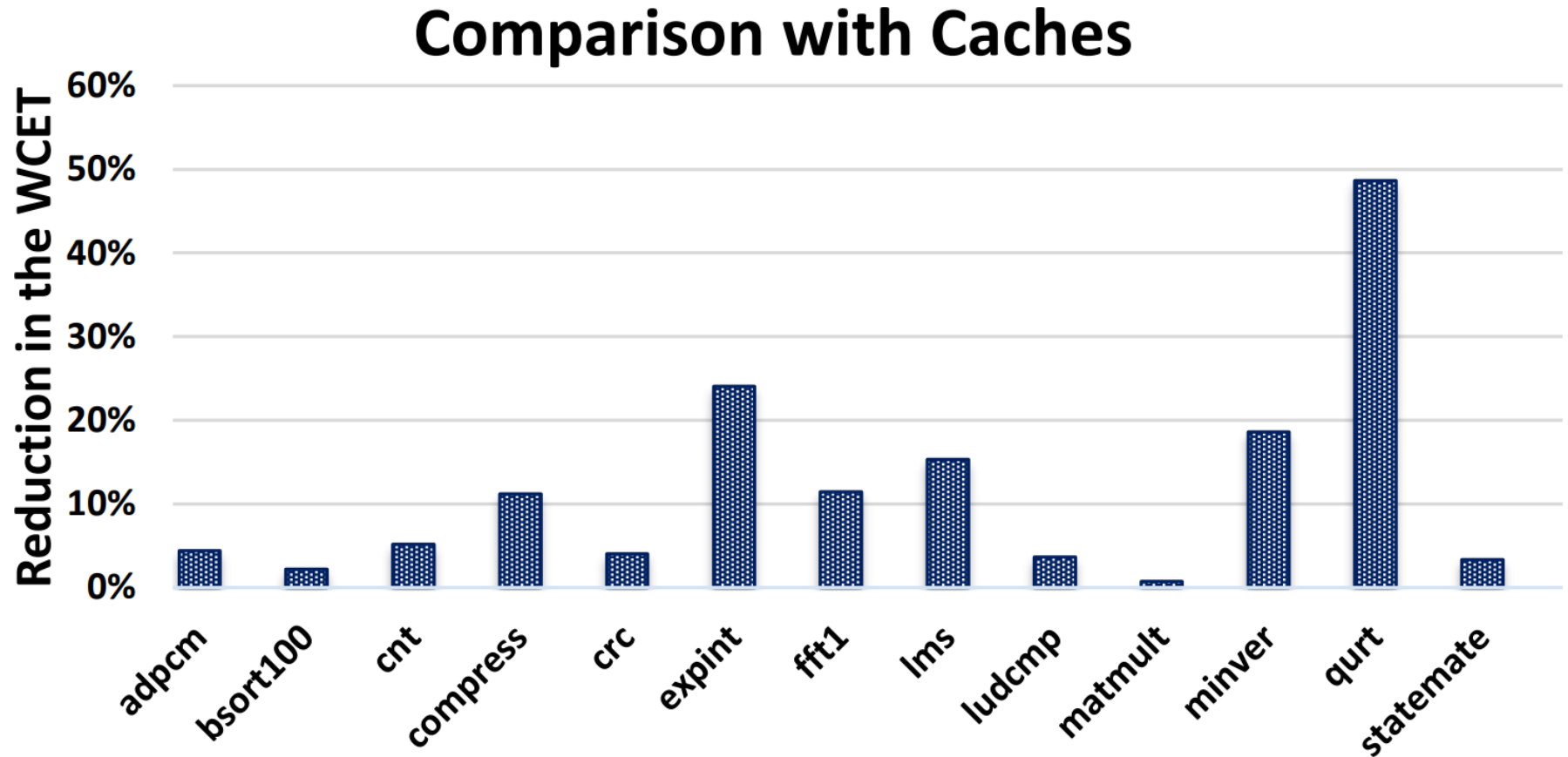
Comparing with Lu et al. (memory config 2)



Comparing with Liu and Zhang (memory config 2)



Comparing with Caches



Conclusion

- ▶ We Proposed a technique to find ***optimal locations*** to perform ***stack management operations*** in order to ***minimize the WCET*** of a given program.
- ▶ We allocate the whole Stack in SPM which simplifies the WCET analysis.
- ▶ Up to 48% reduction in WCET is achieved.